UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS FLORIANÓPOLIS
PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

Henrique Antonio Buzin Vargas

Uma Arquitetura Modular para Ambientes Inter-Névoa

Florianópolis
2025

Henrique Antonio Buzin Vargas

Uma Arquitetura Modular para Ambientes Inter-Névoa

Dissertação submetida ao Programa de Pós-Graduação em Ciência da Computação da Universidade Federal de Santa Catarina para a obtenção do título de mestre em Ciência da Computação.
Orientador: Prof.ª Dr.ª Patricia Della Méa Plentz

Florianópolis
2025

Ficha de identificação da obra

A ficha de identificação é elaborada pelo próprio autor.
Orientações em:
http://portalbu.ufsc.br/ficha

Henrique Antonio Buzin Vargas

Uma Arquitetura Modular para Ambientes Inter-Névoa

O presente trabalho em nível de mestrado foi avaliado e aprovado por banca examinadora composta pelos seguintes membros:

Prof. Dr. Odorico Machado Mendizabal
Universidade Federal de Santa Catarina

Prof.(a) xxxx, Dr(a).
Instituição xxxx

Prof.ª Dr.ª Atslands Rego da Rocha
Universidade Federal do Ceará

Certificamos que esta é a **versão original e final** do trabalho de conclusão que foi julgado adequado para obtenção do título de mestre em Ciência da Computação.

Coordenação do Programa de
Pós-Graduação

Prof.ª Dr.ª Patricia Della Méa Plentz
Orientador

Florianópolis, 2025.

Dedico este trabalho a todos os meus professores,
sendo, meus pais, os maiores entre eles.

AGRADECIMENTOS

Neste momento, sinto uma gratidão imensa por cada pessoa que fez parte desta jornada. Agradeço de coração aos meus pais e à minha namorada, cujos amores incondicionais e apoios constantes foram a base sólida sobre a qual construí meus sonhos e conquistas. Sem vocês, nada disso teria sido possível.

À UFSC (Universidade Federal de Santa Catarina) e a todos os seus membros, expresso meu profundo agradecimento. Cada professor, funcionário e colega desempenhou um papel crucial no meu crescimento acadêmico. Através do ensino de qualidade, das valiosas ferramentas e dos recursos disponibilizados, pude adquirir o conhecimento necessário para realizar este trabalho.

Em especial, minha orientadora Patrícia Della Méa Plentz, você foi uma inspiração e guia ao longo dessa jornada. Sua dedicação, sabedoria e orientação foram fundamentais para a construção desta qualificação. Agradeço por compartilhar seu conhecimento, suas perspectivas e por me motivar a alcançar meu melhor desempenho. Sua generosidade em investir tempo e energia em minha formação é um presente inestimável.

Um agradecimento especial à LexisNexis, em especial a Mauro Marques e Alysson Oliveira. Sua colaboração e apoio foram cruciais para o desenvolvimento deste trabalho. Suas expertise e capacidade analítica, com seu suporte técnico e encorajamento constante, enriqueceram imensamente minha pesquisa. Sou profundamente grato(a) a ambos por sua dedicação e por acreditarem no potencial deste projeto.

A todos vocês, meus pais, minha namorada, minha orientadora e a equipe da LexisNexis, sou profundamente grato(a). Seus ensinamentos, apoio e encorajamento foram a força motriz que me impulsionou a superar desafios e alcançar meus objetivos. Cada palavra de incentivo, cada gesto de apoio e cada momento de aprendizado deixaram uma marca indelével em minha vida acadêmica e pessoal.

Expresso minha sincera admiração e gratidão por cada contribuição que fizeram em minha jornada. Vocês são verdadeiros pilares em minha vida, e sou grato(a) por terem acreditado em mim e por terem compartilhado esse caminho comigo.

”A educação é a arma mais poderosa que
você pode usar para mudar o mundo.”
(Nelson Mandela)

RESUMO

Este trabalho apresenta o desenvolvimento e a avaliação de uma arquitetura modular voltada a ambientes de computação em névoa, projetada para possibilitar a comunicação direta entre diferentes domínios e integrar dispositivos de borda, nós de névoa e a nuvem. A solução proposta foi construída tomando como referência o modelo da OpenFog Consortium (IEEE. . . , 2018) focando nos pilares de escalabilidade, abertura, autonomia, programabilidade e hierarquia, aplicados ao contexto de cidades inteligentes. A proposta incorporou a abstração de protocolos para promover a interoperabilidade entre diferentes tecnologias de comunicação, além de um mecanismo de balanceamento dinâmico de carga e agregação de dados voltado à otimização do uso de recursos e à redução do tráfego de informações para a nuvem. A metodologia experimental envolveu a simulação de dois dias de operação contínua, com dispositivos distribuídos em dois domínios de névoa e geração periódica de dados. Durante os testes, foram analisados indicadores de latência e consumo de memória. Em comparação com outras abordagens, a arquitetura demonstrou menor complexidade e menor consumo computacional. A arquitetura proposta apresentou um consumo total de aproximadamente 374 MiB de memória, enquanto o trabalho de D'Agostino *et al.* (D'Agostino; Longo; Palmieri, 2025) reporta um consumo de aproximadamente 1,2 GB de memória com apenas um contêiner ativo. Esse resultado representa uma redução aproximada de 69% no uso de memória. Como perspectivas futuras, propõe-se a integração de novos protocolos de comunicação, bem como a adoção de mecanismos de autenticação e criptografia ponta a ponta.

Palavras-chave: computação em névoa, arquitetura modular, interoperabilidade, balanceamento de carga, comunicação entre névoas.

ABSTRACT

This work presents the development and evaluation of a modular architecture aimed at fog computing environments, designed to enable direct communication between different domains and integrate edge devices, fog nodes, and the cloud. The proposed solution was built taking the OpenFog Consortium model (IEEE. . . , 2018) as a reference, focusing on the pillars of scalability, openness, autonomy, programmability, and hierarchy, applied to the context of smart cities. The proposal incorporated protocol abstraction to promote interoperability between different communication technologies, in addition to a dynamic load balancing mechanism and data aggregation aimed at optimizing resource usage and reducing information traffic to the cloud. The experimental methodology involved the simulation of two days of continuous operation, with devices distributed across two fog domains and periodic data generation. During the tests, latency and memory consumption indicators were analyzed. Compared to other approaches, the architecture demonstrated lower complexity and lower computational consumption. The proposed architecture presented a total memory consumption of approximately 374 MiB, while the work of D'Agostino *et al.* (D'Agostino; Longo; Palmieri, 2025) reports a memory consumption of approximately 1.2 GB with only one active container. This result represents an approximate reduction of 69% in memory usage. As future perspectives, the integration of new communication protocols is proposed, as well as the adoption of authentication mechanisms and end-to-end encryption.

Keywords: fog computing, modular architecture, interoperability, load balancing, inter-fog communication.

LISTA DE FIGURAS

Figura 1 – Visão geral da arquitetura proposta. 31
Figura 2 – Subcamada de protocolos de um nó de névoa. 34
Figura 3 – Subcamada de processamento de um nó de névoa. 35
Figura 4 – Interface para gerenciamento de contêineres no ambiente de testes. 37
Figura 5 – Arquivos consolidados na *landing zone* do High Performance Computing Cluster (HPCC) *Systems*. 43

LISTA DE QUADROS

LISTA DE TABELAS

Tabela 2 – Quantidade de artigos por palavra-chave e base de dados 24
Tabela 3 – Especificações das máquina utilizada nos testes. 38
Tabela 4 – Latências mínimas, máximas e médias medidas 45
Tabela 5 – Consumo médio de memória por componente 45

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
AWS	Amazon Web Services
CAPES	Coordenação de Aperfeiçoamento de Pessoal de Nível Superior
CoAP	Constrained Application Protocol
CSV	Comma-Separated Values
ECL	Enterprise Control Language
FTP	File Transfer Protocol
GRAPHQL	Graph Query Language
HPCC	High Performance Computing Cluster
HTTP	Hypertext Transfer Protocol
IaaS	Infraestrutura como Serviço
IoT	Internet of Things
JSON	JavaScript Object Notation
L1	Layer 1 Fog
L2	Layer 2 Fog
LSTM	Long Short-Term Memory
MQTT	Message Queuing Telemetry Transport
PaaS	Plataforma como Serviço
RAM	Random Access Memory
REST	Representational State Transfer
RMSE	Root Mean Square Error
ROXIE	Rapid Online XML Inquiry Engine
RTT	Round Trip Time
SaaS	Software como Serviço
SFTP	Secure File Transfer Protocol
SSD	Solid-State Drive
SSH	Secure Shell
TCP	Transmission Control Protocol
TCP/IP	Transmission Control Protocol / Internet Protocol
TI	Tecnologia da Informação
UDP	User Datagram Protocol

SUMÁRIO

1 INTRODUÇÃO . . . 16
1.1 OBJETIVOS . . . 17
1.1.1 Objetivo Geral . . . 17
1.1.2 Objetivos Específicos . . . 17
2 FUNDAMENTAÇÃO TEÓRICA . . . 18
2.1 COMPUTAÇÃO EM NUVEM . . . 18
2.2 COMPUTAÇÃO EM NÉVOA . . . 19
2.3 PROTOCOLOS . . . 20
2.4 *GRAPH QUERY LANGUAGE* (Graph Query Language (GRAPHQL)) 21
2.5 HPCC SYSTEMS . . . 22
3 REVISÃO BIBLIOGRÁFICA . . . 24
3.1 TRABALHOS CORRELATOS . . . 24
3.1.1 Microservice instances selection and load balancing in fog computing using deep reinforcement learning approach . . . 25
3.1.2 Enhancing modular application placement in a hierarchical fog computing: A latency and communication cost-sensitive approach 25
3.1.3 A Scalable Fog Computing Solution for Industrial Predictive Maintenance and Customization . . . 26
3.1.4 Testbed analysis of multi fog architecture for interoperable Internet of Things . . . 26
3.2 COMPARATIVO COM TRABALHOS CORRELATOS . . . 27
4 PROPOSTA . . . 29
4.1 PLANEJAMENTO E DEFINIÇÃO DA ARQUITETURA . . . 29
4.2 IMPLEMENTAÇÃO DOS COMPONENTES . . . 29
4.3 METODOLOGIA DE AVALIAÇÃO . . . 30
4.4 VISÃO GERAL . . . 30
4.4.1 Componentes da Camada de Névoa . . . 31
5 TESTES . . . 36
5.1 CONTEXTUALIZAÇÃO DO PROBLEMA . . . 36
5.2 AMBIENTE DE TESTES . . . 36
5.2.1 Ambiente Físico . . . 37
5.3 TECNOLOGIAS E PROTOCOLOS . . . 38
5.4 DISTRIBUIÇÃO ENTRE NÉVOAS E BALANCEAMENTO DE CARGA 40
5.5 BANCO DE DADOS E LINGUAGEM DE CONSULTA DE DADOS . . 41
5.6 HPCC SYSTEMS NA CAMADA DE NUVEM . . . 42
6 RESULTADOS . . . 44
6.1 VALORES MEDIDOS . . . 44

6.2 DISCUSSÃO DOS RESULTADOS 45
7 CONCLUSÃO 47
Referências 49

1 INTRODUÇÃO

Estamos vivenciando um crescimento exponencial no número de dispositivos conectados à internet, impulsionado principalmente pela expansão da Internet das Coisas (Internet of Things (IoT)). Essa categoria abrange uma ampla gama de equipamentos, como smartphones, relógios inteligentes, sensores ambientais, eletrodomésticos inteligentes e veículos autônomos, entre outros. Esses dispositivos diferem significativamente entre si em termos de capacidade computacional, armazenamento local, autonomia energética e protocolos de comunicação utilizados, o que introduz desafios consideráveis de interoperabilidade e gerenciamento em larga escala.

Paralelamente, cresce a exigência por processamento de dados em fluxo contínuo, especialmente em aplicações de cidades inteligentes. Em redes elétricas inteligentes (*smart grids*), por exemplo, sensores distribuídos coletam continuamente dados sobre consumo e condições operacionais da infraestrutura elétrica urbana, exigindo processamento rápido para permitir respostas quase em tempo real (Khan et al., 2021). Outro exemplo ocorre em sistemas de monitoramento urbano, nos quais sensores instalados na cidade coletam dados ambientais e operacionais para apoiar a gestão de serviços urbanos e a tomada de decisão em tempo próximo ao real (Markakis et al., 2017).

Diante desse cenário, a arquitetura de computação em névoa (*fog computing*) surgiu como uma solução eficaz para suprir a necessidade de processamento e armazenamento mais próximos da borda da rede.

Essa abordagem reduz a latência e melhora a eficiência do sistema, sendo especialmente relevante em aplicações que exigem respostas rápidas e precisas (Markakis et al., 2017; Rahmani et al., 2018). A computação em névoa pode atuar como intermediária entre os dispositivos e a nuvem, realizando um pré-processamento local dos dados antes de enviá-los para a nuvem. Esse processamento próximo à fonte dos dados facilita a integração entre sistemas, reduz o tráfego de rede e diminui a carga sobre os servidores centrais (Tseng; Lin, 2018).

Em uma arquitetura básica de computação em névoa, os dispositivos de borda enviam dados para os equipamentos da camada de névoa, responsáveis pelo processamento local das informações. Após essa etapa, os dados podem ser devolvidos aos dispositivos para suporte a ações imediatas ou encaminhados à camada de nuvem para armazenamento, análise ou processamento adicional, dependendo do contexto da aplicação.

No entanto, a diversidade de dispositivos e protocolos de comunicação ainda representa um desafio para a interoperabilidade em ambientes distribuídos e heterogêneos (Tseng; Lin, 2018; Markakis et al., 2017). A ausência de um padrão universalmente aceito dificulta a comunicação eficiente entre equipamentos heterogêneos.

Com o objetivo de mitigar as limitações de interoperabilidade identificadas, este trabalho apresenta uma arquitetura modular de névoa capaz de estabelecer comunicação entre domínios distintos e gerenciar serviços de forma adaptável e distribuída.

1.1 OBJETIVOS

Nas seções abaixo estão descritos o objetivo geral e os objetivos específicos.

1.1.1 Objetivo Geral

Desenvolver uma arquitetura modular de computação em névoa voltada à comunicação entre diferentes névoas, com foco na integração de novos dispositivos, e na facilitação da interoperabilidade entre protocolos distintos, considerando aplicações cujos requisitos possam ser contemplados por essa abordagem.

1.1.2 Objetivos Específicos

Para atender ao objetivo geral descrito nesta seção, apresentam-se os seguintes objetivos específicos:

- Desenvolver uma arquitetura modular em camadas para a computação em névoa, definindo as funções e responsabilidades de cada camada, com foco na integração de novos dispositivos.
- Projetar e implementar um sistema de comunicação entre névoas, visando facilitar a interoperabilidade e reduzir a latência entre dispositivos que utilizam diferentes padrões e tecnologias, atendendo a aplicações cujos requisitos sejam compatíveis com a arquitetura.
- Desenvolver e integrar mecanismos de conversão de protocolos, permitindo a comunicação entre dispositivos com diferentes padrões e tecnologias.
- Avaliar a eficiência da arquitetura modular proposta por meio da medição de tempo de resposta, latência e quantidade de pacotes com e sem falhas, em simulações que reproduzam requisitos específicos das aplicações.
- Documentar os resultados obtidos e propor diretrizes, fornecendo orientações e melhores práticas para futuras implementações e pesquisas que atendam a requisitos similares.

2 FUNDAMENTAÇÃO TEÓRICA

Neste capítulo, apresentam-se os conceitos fundamentais para a compreensão do contexto desta pesquisa. Desta forma, iniciamos com a conceituação do que é computação distribuída e em névoa e avançamos nos conceitos pertinentes a este trabalho.

2.1 COMPUTAÇÃO EM NUVEM

A computação em nuvem (*Cloud Computing*) é definida como um modelo que permite acesso ubíquo, conveniente e sob demanda, por meio da rede, a um conjunto compartilhado de recursos computacionais configuráveis, como redes, servidores, armazenamento, aplicações e serviços, que podem ser rapidamente provisionados e liberados com mínimo esforço de gerenciamento ou interação com o provedor de serviços (Mell; Grance, 2011). Essa abordagem oferece uma maneira eficiente de fornecer serviços de Tecnologia da Informação (TI) sob demanda, proporcionando flexibilidade, escalabilidade e redução de custos. A computação em nuvem envolve a utilização de *data centers* distribuídos para fornecer esses recursos, permitindo que os usuários acessem e utilizem capacidades computacionais de acordo com suas necessidades, sem a necessidade de investir em hardware próprio (Tanenbaum; Bos, 2015).

Os modelos de serviço da computação em nuvem são geralmente classificados em três categorias principais: Infraestrutura como Serviço (Infraestrutura como Serviço (IaaS)), Plataforma como Serviço (Plataforma como Serviço (PaaS)) e Software como Serviço (Software como Serviço (SaaS)). No modelo IaaS, os provedores de nuvem oferecem recursos virtualizados, como servidores e armazenamento, que os clientes podem configurar e gerenciar conforme suas necessidades. Esse modelo é ideal para empresas que necessitam de controle sobre a infraestrutura e desejam a flexibilidade de escalar seus recursos conforme a demanda. Exemplos de IaaS incluem Amazon Web Services (Amazon Web Services (AWS)) e Microsoft Azure.

No modelo PaaS, a nuvem fornece uma plataforma que permite aos desenvolvedores criar, testar e implantar aplicações sem se preocupar com a gestão da infraestrutura subjacente. Tanenbaum e Bos destacam que o PaaS simplifica o processo de desenvolvimento ao fornecer um ambiente integrado com ferramentas e serviços necessários para a construção de aplicações. Exemplos de PaaS incluem Google App Engine e Heroku.

O modelo SaaS oferece aplicações de software através da internet, eliminando a necessidade de instalação e manutenção de software localmente nos dispositivos dos usuários. As aplicações SaaS são acessíveis via navegador *web*, proporcionando uma experiência de uso simplificada e atualizações automáticas. Exemplos populares de SaaS incluem Google Workspace, Microsoft Office 365 e Salesforce.

Tanenbaum e Bos enfatizam que a computação em nuvem também proporciona vantagens significativas em termos de continuidade de negócios e recuperação de desastres. Ao armazenar dados e executar aplicações em *data centers* distribuídos geograficamente, a nuvem garante que as operações possam ser retomadas rapidamente em caso de falhas locais, aumentando a resiliência e a segurança das operações.

2.2 COMPUTAÇÃO EM NÉVOA

A computação em névoa (*fog computing*) surge como uma extensão do paradigma de computação em nuvem, oferecendo uma solução para limitações de latência e largura de banda observadas em arquiteturas baseadas exclusivamente em nuvem. Diferente da computação em nuvem, onde os dados são processados em *data centers* distantes, a computação em névoa distribui recursos computacionais, de armazenamento e serviços ao longo da borda da rede, aproximando o processamento dos dispositivos finais. Essa abordagem hierárquica envolve dispositivos finais, nós de névoa e a nuvem centralizada, permitindo a distribuição eficiente de tarefas e o processamento local de dados, aspecto essencial para aplicações que exigem respostas rápidas e processamento contínuo de informações (Rahmani et al., 2018).

Entre as principais vantagens da computação em névoa, destaca-se a redução significativa da latência, característica essencial para aplicações que operam com dados em fluxo contínuo. Esse aspecto é particularmente relevante em cenários de cidades inteligentes, nos quais diferentes serviços urbanos dependem da coleta e do processamento rápido de informações provenientes de sensores distribuídos na infraestrutura da cidade, como sistemas de monitoramento de consumo de água e energia, redes elétricas inteligentes e plataformas de gestão urbana. Nesses ambientes, ocorre uma troca constante de dados entre dispositivos, nós de processamento e centros de análise, apoiando a tomada de decisão e o funcionamento eficiente dos serviços urbanos (Khan et al., 2021; Tseng; Lin, 2018).

Além disso, a economia de largura de banda pode ser obtida por meio do pré-processamento e da filtragem de dados realizados localmente, diminuindo a quantidade de informações transmitidas para a nuvem. A segurança e a privacidade dos dados também podem ser aprimoradas quando informações sensíveis são processadas mais próximas de sua origem, reduzindo os riscos associados à transmissão de grandes volumes de dados pela rede (Rahmani et al., 2018). A alta disponibilidade e a resiliência dos serviços também são beneficiadas pela distribuição de recursos em múltiplos nós de névoa, aumentando a robustez do sistema frente a possíveis falhas (Oliveira; Santos; Mendes, 2024).

No contexto da Internet das Coisas (IoT), a computação em névoa revela-se particularmente relevante em aplicações relacionadas a cidades inteligentes. Nesse cenário, a presença de sensores e dispositivos distribuídos pela infraestrutura urbana

gera grandes volumes de dados que precisam ser processados de forma rápida e eficiente. A computação em névoa permite que parte desse processamento ocorra próximo à origem dos dados, reduzindo a dependência de comunicação constante com a nuvem.

Em cidades inteligentes, a névoa possibilita o monitoramento e o controle mais eficientes de diferentes serviços urbanos, como sistemas de gerenciamento de tráfego, redes elétricas inteligentes e plataformas de monitoramento de consumo de água e energia. Sensores distribuídos pela cidade coletam continuamente informações sobre o funcionamento da infraestrutura urbana, enquanto os nós de névoa realizam o processamento inicial desses dados, filtrando, agregando ou analisando as informações antes de encaminhá-las para sistemas centrais ou para a nuvem.

Essa abordagem contribui para melhorar a capacidade de resposta das aplicações urbanas, apoiar a tomada de decisão em tempo quase real e otimizar o uso de recursos da cidade. Além disso, ao distribuir o processamento entre múltiplos nós próximos aos dispositivos, a computação em névoa reduz o volume de dados transmitidos para a nuvem e melhora a eficiência geral da infraestrutura digital das cidades inteligentes.

Apesar das vantagens, a computação em névoa ainda enfrenta desafios relacionados à heterogeneidade dos dispositivos, à complexidade na gestão de recursos distribuídos e à necessidade de padrões interoperáveis. Esses desafios abrem espaço para pesquisas e propostas de soluções que busquem aprimorar o desempenho e a adoção desse paradigma de computação distribuída.

2.3 PROTOCOLOS

Os protocolos de comunicação são fundamentais para o funcionamento das redes de computadores e para a troca eficiente e segura de informações entre dispositivos. Eles definem um conjunto de regras e normas que permitem a comunicação entre diferentes sistemas, sejam eles locais ou distribuídos globalmente (Tanenbaum; Wetherall, 2011). Na computação em névoa, os protocolos são usados para garantir a comunicação eficiente e segura entre dispositivos e servidores locais, permitindo a troca de informações de forma robusta e confiável. A seguir são detalhados quatro protocolos de comunicação de comunicação usados na computação em névoa.

Hypertext Transfer Protocol (HTTP) (*Hypertext Transfer Protocol*) é um protocolo de requisição-resposta usado principalmente para a troca de informações na *web*. Ele opera sobre a camada de transporte Transmission Control Protocol / Internet Protocol (TCP/IP) e permite a comunicação entre clientes (navegadores *web*) e servidores. Os métodos mais comuns incluem GET (para recuperar dados) e POST (para enviar dados). O HTTP é sem estado, o que significa que cada requisição é independente, simplificando o design do servidor, mas pode necessitar de gerenciamento adicional

para manter sessões de usuário (Gourley et al., 2002).

WebSocket é um protocolo de comunicação que proporciona um canal bidirecional e *full-duplex* sobre uma única conexão Transmission Control Protocol (TCP). Diferente do HTTP, que segue um modelo de requisição-resposta, o *WebSocket* permite que dados sejam enviados e recebidos de ambos os lados a qualquer momento após a conexão inicial. Isso o torna ideal para aplicações que necessitam de atualizações em tempo real, como chats, jogos online e sistemas de trading financeiro, oferecendo menor latência e maior eficiência (Wang et al., 2013).

Secure File Transfer Protocol (SFTP) (*Secure File Transfer Protocol*) é um protocolo para transferência segura de arquivos que utiliza criptografia Secure Shell (SSH) para proteger a confidencialidade e integridade dos dados durante a transferência. Diferente do File Transfer Protocol (FTP) tradicional, que transmite dados em texto claro, o SFTP encripta tanto os comandos quanto os dados, proporcionando uma camada adicional de segurança. É amplamente utilizado em ambientes que requerem troca segura de arquivos, como empresas e instituições acadêmicas, onde a proteção dos dados é crucial (Barrett; Silverman; Byrnes, 2005).

Constrained Application Protocol (CoAP) (*Constrained Application Protocol*) é um protocolo projetado para dispositivos restritos e redes de baixa capacidade, típico no contexto da Internet das Coisas (IoT). Baseado no modelo de requisição-resposta do HTTP, CoAP é otimizado para operar sobre User Datagram Protocol (UDP), resultando em menor sobrecarga de comunicação e melhor desempenho em redes de baixa capacidade. CoAP suporta mecanismos de confiabilidade e é interoperável com HTTP, facilitando a integração entre dispositivos IoT e a *web* tradicional (Bormann; Castellani; Shelby, 2012).

2.4 *GRAPH QUERY LANGUAGE* (GRAPHQL)

Graph Query Language (GRAPHQL) é uma linguagem de consulta e manipulação de dados para Application Programming Interface (API)s que foi desenvolvida pelo Facebook em 2012 e lançada como um projeto de código aberto em 2015. Em contraste com o tradicional modelo *Representational State Transfer* (Representational State Transfer (REST)), GRAPHQL permite que os clientes solicitem exatamente os dados de que precisam, sem a necessidade de receber informações excedentes. Essa capacidade de seleção pode reduzir o volume de dados transferidos pela rede, resultando em uma maior eficiência tanto na comunicação quanto no desempenho geral das aplicações (Silveira, 2019).

Uma das principais vantagens do GRAPHQL é sua flexibilidade na recuperação de dados. Enquanto a arquitetura (REST) requer múltiplas requisições para diferentes *endpoints* para compilar dados relacionados, o GraphQL permite que todas essas informações sejam obtidas através de uma única consulta. Isso é possível porque o

cliente especifica exatamente quais campos e relacionamentos deseja recuperar, e o servidor responde com os dados estruturados conforme solicitado. Esse modelo de consulta única não apenas simplifica o processo de desenvolvimento, mas também minimiza o tempo de resposta e a carga na rede.

Além disso, o GRAPHQL facilita a evolução das APIs. No modelo (REST), a adição de novos campos ou a alteração da estrutura de dados pode exigir a criação de novas versões de *endpoints* para garantir a compatibilidade retroativa. Com GraphQL, as APIs podem evoluir de maneira mais fluida. Novos campos podem ser adicionados às respostas sem impactar os clientes existentes, pois cada cliente continua a solicitar apenas os dados específicos de que precisa.

Adicionalmente, a abordagem de GRAPHQL também melhora a experiência de desenvolvimento ao fornecer uma documentação mais clara e precisa. A própria estrutura da linguagem e as ferramentas associadas, como o GraphiQL, permitem que os desenvolvedores explorem e testem as APIs de maneira interativa. Isso contrasta com a documentação tradicional de APIs (REST), que pode ser menos intuitiva e exigir mais esforço para ser mantida e compreendida.

2.5 HPCC SYSTEMS

O *High Performance Computing Cluster* (HPCC) *Systems* é uma plataforma de processamento massivamente paralelo, de código aberto, desenvolvida pela divisão HPCC Systems da LexisNexis Risk Solutions. Seu objetivo é lidar com grandes volumes de dados, desde a ingestão até a entrega de produtos de dados, com tempos de resposta reduzidos. Criado antes da popularização do Hadoop, não se baseia no paradigma *map-reduce*, possuindo uma arquitetura própria (Taylor, 2022).

A plataforma é composta por um conjunto de servidores de infraestrutura responsáveis por funções como compilação de código, gerenciamento distribuído de dados e execução de trabalhos (*workunits*). O processamento é realizado por dois tipos de clusters: Thor e ROXIE.

O Thor é voltado para operações intensivas em grandes conjuntos de dados, realizando tarefas como transformação, limpeza, integração e análise em larga escala. É utilizado como ferramenta de *back office*, transformando dados brutos em produtos finais e executando análises complexas.

O *Rapid Online XML Inquiry Engine* (Rapid Online XML Inquiry Engine (ROXIE)) é destinado a consultas rápidas e atendimento a solicitações de usuários finais, buscando oferecer respostas em tempos muito reduzidos, frequentemente inferiores a um segundo. É, portanto, o componente voltado ao acesso e entrega de dados processados.

Ambos os clusters utilizam a linguagem *Enterprise Control Language* (Enterprise Control Language (ECL)), de natureza declarativa. Nessa abordagem, o progra-

mador especifica o que deve ser feito, enquanto o sistema determina a forma mais eficiente de execução. Isso contribui para simplificar o desenvolvimento e reduzir a possibilidade de erros.

A arquitetura do HPCC *Systems* foi concebida para permitir escalabilidade progressiva e operação integrada, incorporando desde a ingestão e transformação de dados até a disponibilização dos resultados, sem depender de ferramentas externas para compor o ambiente de produção. Por ser de código aberto, pode ser implantado tanto em ambientes físicos com hardware convencional quanto em configurações baseadas em contêineres, adaptando-se às necessidades de diferentes cenários de processamento de dados. No contexto deste trabalho, a HPCC é usada como IaaS fornecendo a infraestrutura necessária para o processamento de dados advindos da estrutura proposta de computação em névoa.

3 REVISÃO BIBLIOGRÁFICA

Para a realização deste trabalho, foram elaboradas pesquisas em duas bases de dados científicas usando um conjunto de critérios que relacionam palavras-chaves, período de busca e outras opções disponíveis nas diferentes bases. As bases de dados pesquisadas foram *Institute of Electrical and Electronics Engineers* (Instituto de Engenheiros Eletricistas e Eletrônicos - IEEE) e a Scopus. As palavras-chave usadas foram: *'fog'* e *'fog' AND 'architecture'*. Foi aplicado um filtro temporal abrangendo os últimos cinco anos, isto é, de 2020 a 2025. A Tabela 2 apresenta a quantidade de publicações encontradas em cada base de dados para os respectivos termos de busca.

Tabela 2 – Quantidade de artigos por palavra-chave e base de dados

Palavra-chave	**Banco de Dados**	**Quantidade de Artigos**
'fog'	Scopus IEEE	34.309 8.524
'fog' AND 'architecture'	Scopus IEEE	12.822 2.502

Fonte: Do autor.

Com base nos dados apresentados, observa-se que a base Scopus possui o maior número de artigos indexados, destacando-se como a principal fonte de publicações sobre o tema. Por outro lado, a base IEEE apresentou uma quantidade inferior de resultados, embora ainda represente um repositório relevante para estudos na área de computação em névoa.

Além disso, foram aplicados os seguintes critérios de inclusão para seleção dos estudos analisados nesta pesquisa:

- Artigos de pesquisa completos;
- Artigos redigidos em língua inglesa;
- Publicações classificadas no sistema Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) Qualis[1] entre os estratos A1 e B2;
- Trabalhos revisados por pares e publicados em conferências ou periódicos científicos reconhecidos.

3.1 TRABALHOS CORRELATOS

Foram lidos títulos e resumo de cento e vinte e cinco artigos, dos quais vinte e cinco foram lidos completamente, e desses foram selecionados quatro que apresentam

[1] A Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) é uma agência do governo brasileiro responsável pela avaliação e fomento da pós-graduação.

propostas semelhantes à proposta deste trabalho e por isso serão detalhados nas subseções seguintes.

3.1.1 Microservice instances selection and load balancing in fog computing using deep reinforcement learning approach

Os autores propõem uma abordagem para a seleção de instâncias de microsserviços e para o balanceamento de carga em ambientes de computação em névoa, empregando técnicas de *Deep Reinforcement Learning*. O estudo parte do princípio de que aplicações da Internet das Coisas podem ser estruturadas de forma modular, utilizando arquiteturas baseadas em microsserviços implantados próximos à borda da rede. Essa configuração, denominada computação *fog* nativa, busca superar as limitações de latência e de custo de comunicação dos modelos puramente em nuvem ao permitir o processamento distribuído de tarefas entre dispositivos, nós de névoa e servidores centrais (Boudieb; Ayed; Harous, 2024).

No modelo proposto, cada aplicação é composta por um conjunto de microsserviços interdependentes organizados em planos de serviço. O desafio central é selecionar esses planos de forma dinâmica e eficiente em ambientes muito variáveis, nos quais a carga de trabalho e as condições de rede mudam constantemente. Para isso, os autores apresentam o método *Microservice Instances Selection Policy*, que utiliza aprendizado por reforço profundo para otimizar a alocação de microsserviços, reduzindo atrasos e distribuindo a carga de forma equilibrada entre os nós disponíveis. O algoritmo incorpora mascaramento de ações, mapeamento adaptativo de ações e um esquema de *experience replay* ajustado para melhorar a eficiência e a estabilidade do treinamento.

Os resultados experimentais indicam que a política proposta reduz a taxa média de falhas em até sessenta e cinco por cento e melhora o balanceamento de carga em cerca de quarenta e cinco por cento em relação a algoritmos convencionais. O trabalho reforça a importância da colaboração entre névoa e nuvem para garantir escalabilidade, desempenho e resiliência em aplicações sensíveis ao tempo.

3.1.2 Enhancing modular application placement in a hierarchical fog computing: A latency and communication cost-sensitive approach

Neste trabalho é proposto uma estratégia de posicionamento de aplicações modulares em uma arquitetura hierárquica de computação em névoa, denominada *Least Impact X*, abreviada como *LI X*. O estudo aborda desafios de alocação de recursos em cenários com vários níveis de névoa, buscando reduzir o tempo de resposta e o custo de comunicação em aplicações sensíveis à latência. A motivação é evitar o esgotamento de recursos nas camadas mais próximas da borda quando requisitos es-

pecíficos de uma aplicação são priorizados sem considerar o efeito global na hierarquia (Oliveira; Santos; Mendes, 2024).

A proposta distribui os módulos entre os níveis de névoa de forma a minimizar o impacto sobre recursos críticos próximos à borda e, ao mesmo tempo, reduzir o tempo de resposta e o consumo de largura de banda. O algoritmo considera métricas de qualidade de serviço para equilibrar a utilização de recursos e evitar ociosidade em níveis superiores. Em simulações no *iFogSim*, a abordagem supera alternativas da literatura ao apresentar menor latência média e menor custo de comunicação, além de uso mais eficiente de recursos computacionais.

3.1.3 A Scalable Fog Computing Solution for Industrial Predictive Maintenance and Customization

É apresentado uma solução de computação em névoa para manutenção preditiva industrial e para customização de sistemas de Internet das Coisas industriais. A arquitetura é modular e flexível e combina sensores Nicla Sense ME, um concentrador em Raspberry Pi e um modelo Long Short-Term Memory (LSTM) para análise preditiva em tempo quase real. O diferencial é um ambiente *sandbox* que permite integrar modelos de aprendizado de máquina de forma isolada, preservando a segurança e a estabilidade do núcleo da plataforma. A solução utiliza Docker para encapsular módulos e escalar para dezenas de nós sensores sem degradação perceptível de desempenho, adotando ainda práticas de segurança como autenticação multifatorial, criptografia e detecção de anomalias (D'Agostino; Longo; Palmieri, 2025).

Nos experimentos, o sistema apresentou erro quadrático médio baixo e alta acurácia em laboratório e manteve desempenho consistente em ambiente industrial. O erro quadrático médio, também referido como Root Mean Square Error (RMSE), mede a diferença média entre valores previstos e valores observados e quanto menor melhor. A acurácia indica a proporção de previsões corretas em relação ao total. Os resultados reforçam a importância de integrar técnicas de inteligência artificial em ambientes industriais com recursos restritos para reduzir paradas, otimizar uso de recursos e permitir personalização.

3.1.4 Testbed analysis of multi fog architecture for interoperable Internet of Things

Os autores investigam uma arquitetura multinévoa com interoperabilidade semântica para aplicações em tempo real, implementada e avaliada em ambiente experimental. O objetivo é permitir que nós de névoa troquem informações e serviços de forma direta e eficaz, reduzindo a dependência de processamento em nuvem. A proposta desloca para a névoa funções usuais do modelo semântico, como filtragem,

anotação e mapeamento, o que permite realizar parte do processamento localmente e diminui o trânsito de dados para servidores distantes (Rahman; Hussain, 2024).

No protótipo, dispositivos Raspberry Pi 2 Model B executam semântica local, armazenamento e serviços. Quando um nó está sobrecarregado, ele avalia se vale a pena repassar a tarefa para outro nó vizinho com base em tempo e energia estimados. Caso nenhum candidato seja melhor, a tarefa é encaminhada para a nuvem. A arquitetura utiliza duas camadas, chamadas Layer 2 Fog (L2) *fog* e Layer 1 Fog (L1) *fog*. A primeira atua como *gateway* para sensores e atuadores usando 6LoWPAN e a segunda concentra serviços e banco de dados com Apache e MariaDB. Há ainda sincronização hierárquica baseada em identificadores e um mecanismo de controle de fila que define quando iniciar a transferência de tarefas.

Os experimentos mostram que o tempo médio de ida e volta de uma mensagem, conhecido como *Round Trip Time* ou Round Trip Time (RTT), foi reduzido em cerca de trinta e sete por cento em comparação ao uso direto da nuvem. O RTT é o intervalo total que uma solicitação leva para ir do dispositivo ao destino e retornar com a resposta e é um indicador direto de responsividade. No estudo, valores médios passaram de aproximadamente duzentos e oitenta milissegundos na nuvem para aproximadamente cento e três milissegundos com a arquitetura multinévoa. Também houve redução de cerca de doze por cento no tempo de execução de tarefas quando a transferência entre nós de névoa estava ativa. O tempo de execução é o período entre o envio de uma tarefa e a conclusão do processamento. Os autores ainda registram diminuição média de dez por cento no consumo de energia dos nós em relação ao funcionamento sem transferência entre nós. O consumo de energia é a quantidade de energia elétrica utilizada pelos dispositivos para realizar as tarefas e é crucial em cenários com recursos limitados.

Esses ganhos são explicados pelo processamento local de parte da semântica, pela divisão de carga entre nós próximos e pelo envio à nuvem apenas quando há real vantagem.

3.2 COMPARATIVO COM TRABALHOS CORRELATOS

Conforme apresentado nas subseções anteriores, os trabalhos analisados exploram diferentes perspectivas da computação em névoa, cada um contribuindo de maneira específica para o avanço da área. De modo geral, observa-se que as propostas anteriores tratam de aspectos pontuais, como otimização de desempenho, posicionamento de módulos e modularidade aplicada a contextos específicos, enquanto o presente trabalho reúne e amplia esses conceitos, com ênfase na interoperabilidade e comunicação entre domínios distintos de névoa.

O trabalho de Boudieb et al. (Boudieb; Ayed; Harous, 2024) propõe o uso de aprendizado por reforço profundo para selecionar instâncias de microsserviços e reali-

zar o balanceamento de carga de forma adaptativa. A abordagem demonstra eficiência na redução de falhas e na melhoria da distribuição de tarefas, mas permanece restrita a um único domínio de névoa, sem prever cooperação entre diferentes nós. Por sua vez, Oliveira et al. (Oliveira; Santos; Mendes, 2024) apresentam uma estratégia hierárquica para o posicionamento de aplicações modulares, buscando reduzir a latência e o custo de comunicação. Embora eficiente na alocação de módulos, a proposta não contempla mecanismos de interoperabilidade entre diferentes camadas ou arquiteturas de névoa.

A solução de D'Agostino, Longo e Palmieri (D'Agostino; Longo; Palmieri, 2025), voltada à manutenção preditiva industrial, introduz modularidade com o uso de contêineres e integração de aprendizado de máquina em tempo quase real. Essa proposta apresenta alta escalabilidade e segurança, mas limita-se ao contexto industrial e não aborda a colaboração entre domínios de névoa. Já Rahman e Hussain (Rahman; Hussain, 2024) propõem uma arquitetura multinévoa com interoperabilidade semântica, permitindo que nós de névoa colaborem com base em métricas como tempo de resposta e consumo de energia. Apesar dos avanços em comunicação distribuída, o modelo ainda depende de mecanismos fixos de decisão e não incorpora uma estrutura modular interna aos nós.

A arquitetura proposta neste trabalho diferencia-se por integrar múltiplas camadas modulares dentro de cada nó de névoa, combinando flexibilidade estrutural, interoperabilidade e cooperação entre domínios. Cada camada possui responsabilidades bem definidas: a subcamada de protocolos realiza a comunicação com diferentes dispositivos e formatos de dados; a subcamada de processamento gerencia o fluxo interno, realiza o armazenamento temporário e coordena as tarefas; e a camada de serviço executa aplicações configuráveis que processam e preparam os dados para as etapas seguintes. Essa organização favorece a manutenção, a escalabilidade e a integração eficiente de novos serviços, permitindo que partes específicas do sistema sejam adaptadas sem comprometer o funcionamento geral.

Além da modularidade interna, outro diferencial importante é a comunicação entre domínios de névoa, que permite o balanceamento cooperativo de carga e o redirecionamento de solicitações entre diferentes regiões conforme a disponibilidade de recursos. O nó primário atua como ponto de entrada e registro dos dispositivos conectados, garantindo distribuição equilibrada e continuidade de operação.

Com base nesses conceitos, o próximo capítulo apresenta em detalhes a arquitetura modular proposta, descrevendo suas camadas, fluxos de comunicação e mecanismos de integração entre névoas, evidenciando como essa estrutura supera as limitações identificadas nos trabalhos correlatos.

4 PROPOSTA

Este capítulo apresenta a arquitetura modular de computação em névoa proposta neste trabalho. Inicialmente, apresenta-se uma visão geral da solução, contextualizando sua organização e o fluxo de dados entre os diferentes níveis do sistema. Em seguida, são descritas as camadas arquiteturais, os componentes da camada de névoa, a organização interna dos nós e os mecanismos de comunicação entre domínios.

4.1 PLANEJAMENTO E DEFINIÇÃO DA ARQUITETURA

O desenvolvimento deste trabalho foi iniciado com uma análise do estado da arte em computação em névoa, comunicação entre névoas e interoperabilidade de protocolos. Essa etapa permitiu identificar abordagens já utilizadas na literatura, bem como lacunas relacionadas à integração entre diferentes domínios de névoa e à interoperabilidade entre componentes distribuídos.

Com base nos resultados dessa análise, foram definidos os requisitos funcionais e estruturais que orientaram a construção da arquitetura proposta. A partir desses requisitos, foi elaborada uma arquitetura modular, na qual são especificados os papéis de cada tipo de nó: nó primário, nós de névoa e nó agregador, bem como a organização interna dos componentes responsáveis pelo processamento e pela comunicação entre os diferentes níveis do sistema.

Nessa fase também foram definidos os mecanismos de comunicação e as interações entre nós pertencentes a diferentes domínios de névoa, estabelecendo as bases para a coordenação, o encaminhamento de requisições e o fluxo de dados dentro da arquitetura proposta.

4.2 IMPLEMENTAÇÃO DOS COMPONENTES

A implementação foi conduzida em etapas, de forma a permitir testes incrementais e ajustes contínuos:

1. Desenvolvimento do nó primário, com funções de registro, balanceamento e encaminhamento de requisições.
2. Implementação dos nós de névoa, estruturados em camadas para suportar diferentes protocolos, processamento interno e execução de serviços.
3. Construção do nó agregador, responsável pela consolidação e envio de dados à nuvem.

4.3 METODOLOGIA DE AVALIAÇÃO

Após a implementação, foram realizados testes com foco em:

- **Interoperabilidade**: avaliar a capacidade da arquitetura de lidar com diferentes protocolos de comunicação.
- **Balanceamento de carga**: avaliar o comportamento do nó primário na distribuição das requisições entre os nós de névoa, incluindo cenários de encaminhamento para outras névoas.
- **Agregação de dados**: analisar a consolidação realizada pelo nó agregador e seu impacto na redução de requisições à nuvem.

4.4 VISÃO GERAL

A arquitetura proposta está organizada de forma hierárquica, distribuindo o processamento e o fluxo de dados entre diferentes níveis do sistema. Essa organização considera a coleta de informações na borda, o processamento intermediário na camada de névoa e o armazenamento e tratamento em maior escala na camada de nuvem.

A divisão estrutural da arquitetura é composta por três camadas principais: camada de dispositivos, camada de névoa e camada de nuvem. Essa separação estabelece níveis distintos dentro do sistema, permitindo descrever o fluxo de dados desde sua origem até sua consolidação final.

A camada de dispositivos corresponde ao nível mais próximo do ambiente físico. Ela é composta por sensores, medidores inteligentes, câmeras e outros equipamentos responsáveis pela geração de dados. Esses dispositivos podem operar com diferentes tecnologias e protocolos de comunicação, conforme sua finalidade e fabricante. Nesse nível ocorre a captura das informações que serão encaminhadas às camadas superiores da arquitetura.

A camada de névoa ocupa o nível intermediário do sistema. Ela recebe os dados provenientes dos dispositivos e realiza o processamento local dentro de um domínio específico. Nessa camada estão localizados os elementos responsáveis pela coordenação das requisições, pela execução do pré-processamento e pela organização do fluxo interno de informações. Cada domínio de névoa é composto por um nó primário, responsável pela coordenação, por nós de névoa que executam o processamento local e por um nó agregador que realiza a consolidação dos dados antes do envio à nuvem.

A camada de nuvem representa o nível superior da arquitetura e atua como ponto central para processamento e gerenciamento de dados em maior escala. Ela recebe as informações já pré-processadas na camada de névoa, especialmente aquelas consolidadas pelo nó agregador, e as mantém disponíveis para consultas, análises e processamento posterior.

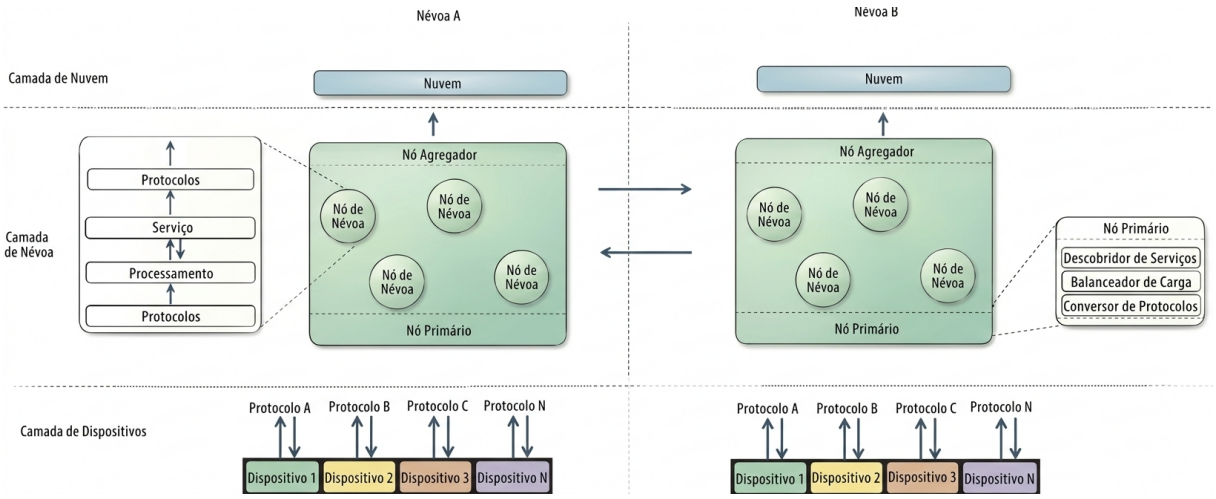
Nesse nível encontram-se recursos computacionais destinados ao tratamento de grandes volumes de dados e à execução de tarefas que podem demandar maior capacidade de processamento, como análises estatísticas, aprendizado de máquina ou outros métodos computacionais aplicados conforme o contexto da aplicação.

Além do processamento, a camada de nuvem é responsável pela manutenção de armazenamento persistente, possibilitando a preservação de dados históricos que podem ser utilizados em análises futuras, auditorias ou estudos complementares. Dessa forma, essa camada complementa o processamento realizado na névoa ao concentrar atividades que exigem maior escala de armazenamento e capacidade computacional.

O fluxo de dados inicia nos dispositivos, é encaminhado à camada de névoa para processamento intermediário e, após consolidação, segue para a camada de nuvem. A estrutura descrita é apresentada de forma resumida na Figura 1, que representa a arquitetura como um todo, incluindo suas camadas arquiteturais e os principais componentes presentes em cada domínio de névoa.

A explicação apresentada até o momento tem caráter geral e introdutório. Nas seções seguintes, cada camada arquitetural, bem como os componentes da camada de névoa e a organização interna dos nós, são descritos de maneira detalhada.

Figura 1 – Visão geral da arquitetura proposta.



Fonte: Do autor.

4.4.1 Componentes da Camada de Névoa

A camada de névoa é composta por três elementos principais: o nó primário, os nós de névoa e o nó agregador. Esses componentes atuam de forma coordenada dentro de cada domínio, participando do fluxo de dados e da organização do processamento intermediário entre dispositivos e nuvem.

O nó primário atua como ponto de entrada do domínio de névoa. Ele recebe dados provenientes dos dispositivos de borda e distribui as requisições entre os nós de névoa disponíveis, considerando a especialização de cada um. Dessa forma, as requisições são direcionadas conforme o tipo de dado ou serviço associado.

Quando um novo nó é iniciado, ele realiza seu registro junto ao nó primário, informando sua especialização, como monitoramento de água ou de energia.

Após o registro, o nó envia atualizações periódicas em intervalos regulares para indicar que permanece ativo. Caso essas atualizações deixem de ser recebidas por um determinado período, o nó é considerado inativo e pode ser removido do conjunto de nós disponíveis no domínio.

A distribuição de carga segue uma lógica semelhante ao *Round Robin*, um algoritmo clássico de balanceamento no qual as requisições são distribuídas sequencialmente entre os nós disponíveis. Diferentemente do *Round Robin* tradicional, que apenas alterna entre os nós sem considerar suas características ou funções, o modelo proposto respeita a especialização de cada nó. Dessa forma, as requisições são encaminhadas apenas para nós capazes de processá-las, garantindo que cada nó atue dentro de sua área de atuação e processe uma requisição por vez. Durante o funcionamento do sistema, o nó primário monitora continuamente o número de dispositivos conectados e o volume de requisições associadas ao domínio.

Com base nessas informações, o nó primário pode adotar duas estratégias de redistribuição de carga. A primeira ocorre dentro do próprio domínio de névoa, quando o nó primário identifica que um dos nós possui maior quantidade de dispositivos associados ou maior volume de requisições. Nesse caso, parte dos dispositivos pode ser redistribuída entre outros nós disponíveis do mesmo domínio, buscando manter um equilíbrio na utilização dos recursos. Essa redistribuição considera também a especialização de cada nó. Por exemplo, dispositivos responsáveis pelo monitoramento de água são direcionados apenas para nós especializados nesse tipo de serviço, enquanto dispositivos relacionados ao consumo de energia são encaminhados para nós correspondentes. Dessa forma, quando existem múltiplos nós com a mesma especialização, como vários nós responsáveis pelo processamento de dados de água, os dispositivos podem ser distribuídos entre eles para evitar sobrecarga em um único nó e manter um equilíbrio na execução dos serviços.

A segunda estratégia ocorre entre diferentes domínios de névoa. Caso o nó primário identifique que a capacidade de processamento local está próxima do limite ou que existe desequilíbrio na disponibilidade de especializações, ele pode consultar outros nós primários para verificar a disponibilidade de recursos em seus domínios. Com base nessas informações, novas requisições podem ser encaminhadas para outro domínio de névoa com maior capacidade disponível.

Ao ser inicializado, o nó primário informa os demais nós primários sobre sua

ativação e comunica a quantidade de nós especializados sob sua coordenação. Em situações de desequilíbrio, pode consultar outros domínios para verificar disponibilidade de recursos. Caso identifique sobrecarga, envia uma mensagem de *broadcast* aos demais nós primários solicitando informações atualizadas sobre utilização. Com base nas respostas recebidas, novas requisições podem ser redirecionadas para domínios com maior disponibilidade de recursos.

Em situações de indisponibilidade do nó primário de um domínio, outro nó primário pode assumir temporariamente a coordenação do domínio afetado. Nesse caso, o nó primário ativo envia mensagens de controle aos dispositivos conectados, instruindo-os a redirecionar suas transmissões para o novo nó primário disponível. Esse mecanismo permite manter o fluxo de dados mesmo na ausência do nó primário originalmente responsável pelo domínio.

Os nós de névoa são responsáveis pelo processamento local das informações recebidas do nó primário. Cada nó executa funções específicas de acordo com sua especialização, tratando os dados antes de sua consolidação ou encaminhamento.

Embora a arquitetura esteja organizada em três camadas principais (dispositivos, névoa e nuvem), o nó de névoa possui uma organização interna própria. Para distinguir essa organização das camadas arquiteturais, os níveis internos do nó são denominados, neste trabalho, como subcamadas.

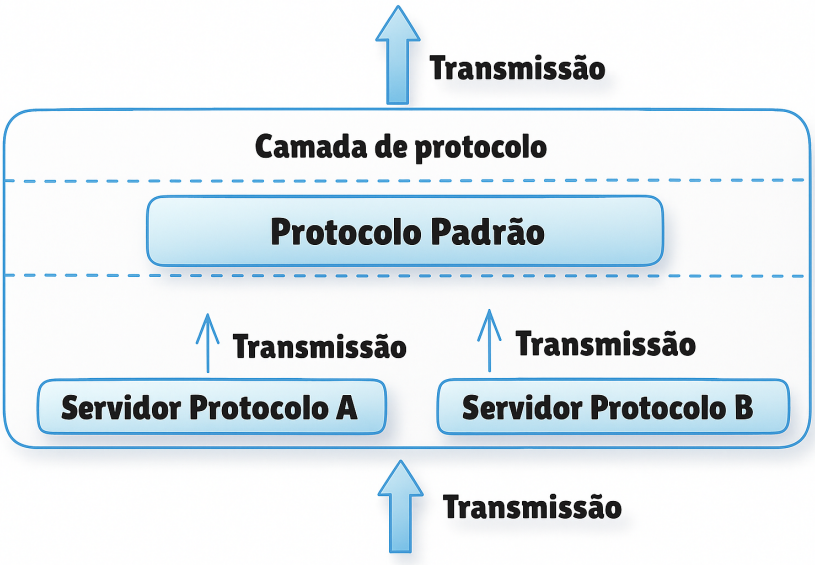
Internamente, os nós de névoa são organizados em uma estrutura composta por três subcamadas funcionais: protocolos, processamento e serviços. Essa divisão organiza as responsabilidades internas de funcionamento do nó.

A subcamada de protocolos é responsável por receber os dados e requisições provenientes do nó primário. Quando necessário, realiza conversões de protocolo, encaminhando as informações para um padrão interno utilizado pelo nó. O protocolo padrão adotado internamente nessa subcamada é orientado a mensagens e não exige o estabelecimento de sessão a cada comunicação, permitindo trocas de dados de forma mais eficiente e com menor sobrecarga de conexão. A Figura 2 apresenta a organização dessa subcamada.

A subcamada de processamento atua como núcleo de coordenação interna do nó. Ela organiza o fluxo de informações entre as subcamadas do nó, mantendo armazenamento temporário dos dados e executando funções relacionadas ao controle interno do processamento. Nessa subcamada também está presente um banco de dados interno responsável por armazenar configurações do domínio de névoa e manter dados temporários utilizados durante o processamento.

Para permitir o acesso estruturado a essas informações, a subcamada disponibiliza uma linguagem de consulta de dados, utilizada pelas demais partes do sistema para recuperar ou registrar informações no banco interno. Esse mecanismo permite que a subcamada de serviços acesse apenas os dados necessários para sua execu-

Figura 2 – Subcamada de protocolos de um nó de névoa.



Fonte: Do autor.

ção, mantendo a organização do fluxo interno de informações.

Além disso, essa subcamada mantém um servidor responsável pela descoberta de serviços disponíveis no nó, permitindo que aplicações ou componentes internos identifiquem funcionalidades presentes no domínio. Também controla as informações associadas ao registro do nó junto ao nó primário.

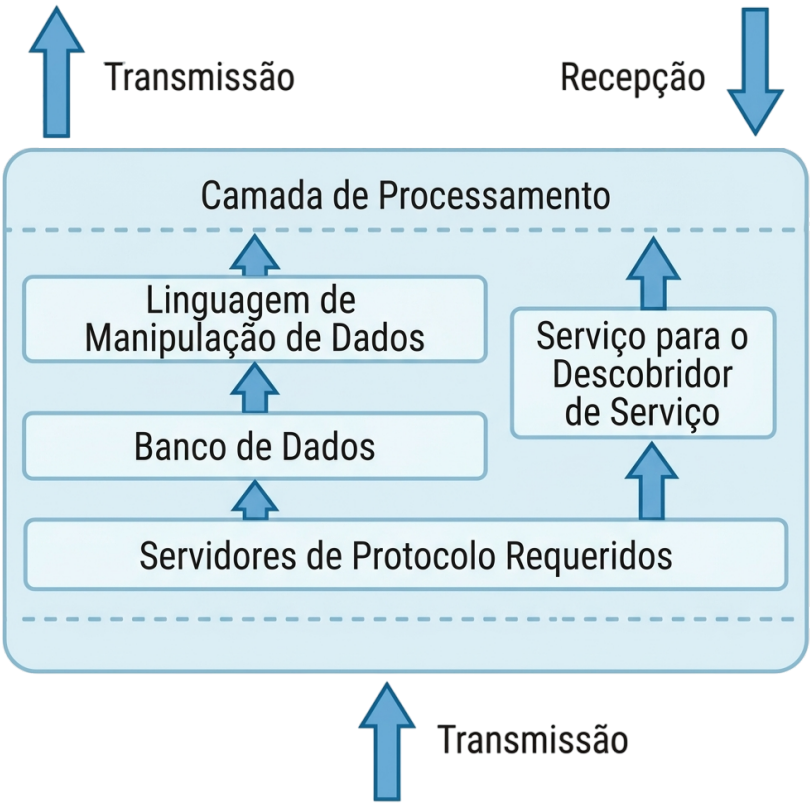
A comunicação entre a subcamada de processamento e a subcamada de protocolos ocorre por meio de um protocolo interno padrão, utilizado para encaminhar requisições e respostas dentro do nó. Dessa forma, a subcamada de processamento prepara os dados e os encaminha à subcamada de protocolos, que é responsável pela comunicação externa com outros componentes do sistema. A Figura 3 apresenta os principais elementos dessa subcamada.

Por fim, a subcamada de serviços executa a lógica de aplicação definida para o domínio. Utilizando a linguagem de consulta disponibilizada pela subcamada de processamento, essa subcamada obtém os dados necessários, realiza o tratamento correspondente e devolve o resultado à subcamada de processamento, que ajusta a saída para envio pela subcamada de protocolos.

O nó agregador recebe os dados já pré-processados pelos nós de névoa e realiza a consolidação antes do envio à nuvem. Sua função consiste em reunir informações provenientes de diferentes nós dentro do domínio.

Quando necessário, o agregador realiza conversões de formato ou protocolo para manter compatibilidade com a infraestrutura em nuvem. Ao receber os dados, combina as informações em um único arquivo estruturado, que pode incluir médias,

Figura 3 – Subcamada de processamento de um nó de névoa.



Fonte: Do autor.

totais ou agrupamentos, conforme o contexto da aplicação.

Após a consolidação, os dados são encaminhados para a camada de nuvem, onde permanecem disponíveis para armazenamento e análises posteriores. Com base nessa arquitetura e na organização de seus componentes, foram conduzidos experimentos para observar o comportamento do sistema e avaliar o funcionamento dos mecanismos propostos. Os resultados obtidos a partir desses experimentos são apresentados e discutidos no capítulo seguinte.

5 TESTES

Este capítulo descreve os procedimentos de teste adotados para avaliar a arquitetura proposta, abrangendo ambiente, cenários, métricas, coleta e análise dos dados.

5.1 CONTEXTUALIZAÇÃO DO PROBLEMA

Para avaliar a arquitetura proposta, foi adotado um cenário inspirado em aplicações de cidades inteligentes, com foco na gestão de recursos de água e energia. Nesse contexto, foram considerados medidores inteligentes que enviam dados de consumo para uma camada de névoa, onde ocorre o pré-processamento das informações antes de seu encaminhamento à nuvem.

Os dados utilizados para simulação foram obtidos a partir de conjuntos públicos, sendo um relacionado ao consumo residencial de energia elétrica (Hernández et al., s.d.) e outro referente a medições de consumo de água (Hellenic Data Service, s.d.).

Antes da utilização nos testes, ambos os conjuntos de dados passaram por um processo de pré-processamento que incluiu a remoção de informações não essenciais e a agregação das medições em intervalos horários. Esse tratamento visou manter apenas os elementos relevantes para simular a operação contínua dos dispositivos, aproximando o comportamento dos dados ao fluxo esperado em um ambiente real de monitoramento urbano.

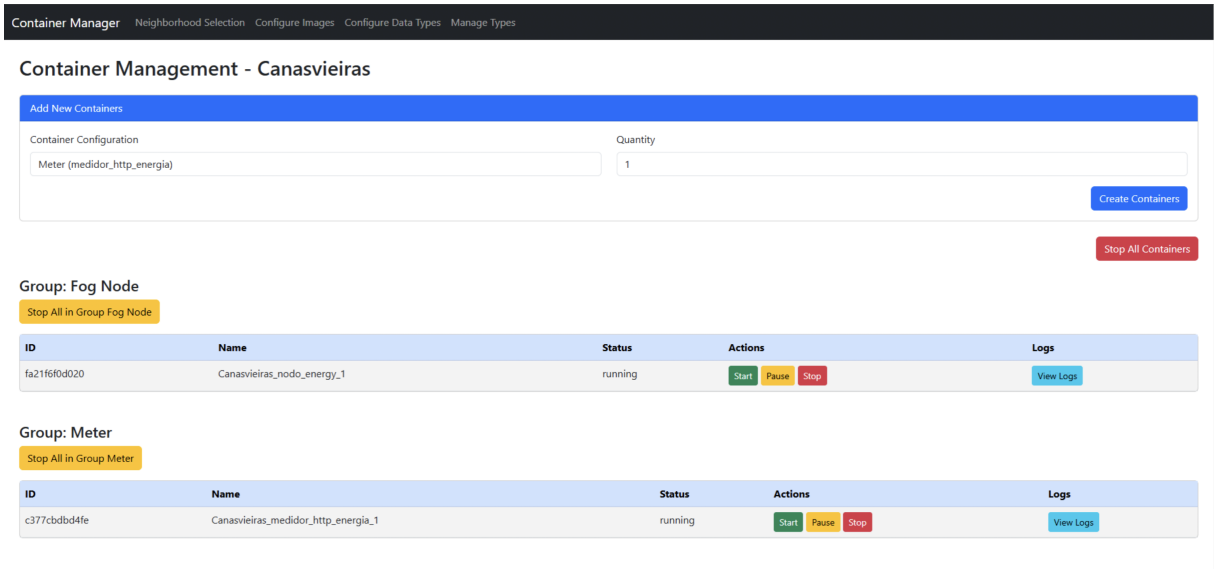
5.2 AMBIENTE DE TESTES

O ambiente de testes foi configurado para representar um cenário de cidade inteligente, no qual cada domínio de névoa corresponde a um bairro distinto. Cada névoa pode conter um nó primário, uma quantidade variável de nós de névoa, e um nó agregador.

A implantação e o gerenciamento desses componentes foram realizados por meio de uma *interface web* desenvolvida para este trabalho, que interage diretamente com o Docker para criar e controlar as instâncias. Essa interface permite selecionar o bairro (névoa) no qual os componentes serão implantados, definir o tipo de contêiner e a quantidade de instâncias, além de oferecer controle individual ou em grupo para iniciar, pausar, parar e visualizar logs.

A Figura 4 apresenta a tela de gerenciamento de contêineres para o bairro Canasvieiras, localizado no município de Florianópolis, no estado de Santa Catarina. Nela, observa-se a criação de um nó de névoa e de um medidor de energia, ambos em execução. O painel agrupa as instâncias por tipo de componente, permitindo visualizar rapidamente seu estado e realizar ações sobre cada grupo ou instância individual.

Figura 4 – Interface para gerenciamento de contêineres no ambiente de testes.



Fonte: Do autor.

5.2.1 Ambiente Físico

Os experimentos foram realizados em quatro máquinas físicas, cada uma com uma função específica dentro da arquitetura distribuída proposta. Cada máquina executou contêineres Docker gerenciados por uma *interface web*, e todas as comunicações entre elas ocorreram por meio de túneis seguros configurados para interligar redes distintas.

A Tabela 3 apresenta as especificações de hardware e a função de cada equipamento. As máquinas PC-01 e PC-02, localizadas em Florianópolis, operaram em uma mesma rede local compartilhada. Elas foram configuradas como nós de névoa completos, denominados Névoa A e Névoa B, cada uma composta por um nó primário, um nó agregador e uma quantidade variável de nós de névoa.

A máquina PC-04, situada em Curitiba e conectada por uma rede independente, foi utilizada para hospedar os dispositivos medidores, simulando o envio de dados provenientes de diferentes origens geográficas.

A máquina PC-03, também localizada em Florianópolis, foi dedicada à camada de nuvem, executando a aplicação HPCC *Systems*, o cluster de alto desempenho utilizado neste trabalho. Essa camada recebeu as informações processadas pelos nós de névoa e centralizou o armazenamento e a consolidação dos resultados.

Tabela 3 – Especificações das máquina utilizada nos testes.

PC-01	
Cidade	Florianópolis
Rede	Compartilhada com o PC-02
Função	Névoa A
Processador	Intel Core i7-7700K @ 4.20GHz
Memória (Random Access Memory (RAM))	32 GB DDR4
Armazenamento	2 TB Solid-State Drive (SSD)
Sistema Operacional	Windows 11 64-bit
PC-02	
Cidade	Florianópolis
Rede	Compartilhada com o PC-01
Função	Névoa B
Processador	Intel Core i7-7500U @ 2.70GHz
Memória (RAM)	36 GB DDR4
Armazenamento	480 GB SSD
Sistema Operacional	Windows 11 64-bit
PC-03	
Cidade	Florianópolis
Rede	Independente
Função	Nuvem (HPCC *Systems*)
Processador	Intel Core i3 5005U @ 2.0GHz
Memória (RAM)	8GB DDR3
Armazenamento	240 GB SSD
Sistema Operacional	Windows 10 64-bit
PC-04	
Cidade	Curitiba
Rede	Independente
Função	Medidores
Processador	AMD Ryzen 5 3600 @ 3.60GHz
Memória (RAM)	32 GB DDR4
Armazenamento	1TB SSD
Sistema Operacional	Windows 10 64-bit

Fonte: Do autor.

5.3 TECNOLOGIAS E PROTOCOLOS

Os experimentos consideram dispositivos medidores responsáveis pela geração de dados de consumo de água e energia. Esses dispositivos representam sensores de borda que enviam medições para a arquitetura proposta.

Para a geração dos dados utilizados pelos medidores, foram empregados conjuntos de dados públicos referentes ao consumo de energia elétrica e de água. O conjunto de dados utilizado para representar o consumo de energia elétrica foi obtido

a partir de um dataset público disponibilizado pela UCI Machine Learning Repository[1]. Já os dados utilizados para representar o consumo de água foram obtidos a partir de um dataset público disponibilizado pelo Hellenic Data Service[2]. Ambos os conjuntos de dados passaram por um processo de pré-processamento antes de serem utilizados nos experimentos. Esse processo incluiu a padronização da frequência temporal dos registros para intervalos de uma hora, permitindo que os dados representassem um comportamento uniforme de envio de medições pelos dispositivos. Além disso, atributos e registros que não eram necessários para os experimentos foram removidos, mantendo apenas as informações relevantes para os testes realizados na arquitetura proposta.

Para avaliar a interoperabilidade entre os componentes da arquitetura, os dispositivos medidores foram desenvolvidos para operar utilizando diferentes protocolos de comunicação. Foram considerados os protocolos HTTP e CoAP, permitindo observar o comportamento do sistema ao receber dados provenientes de diferentes tecnologias de comunicação.

Para representar o funcionamento desses dispositivos no ambiente experimental, foram desenvolvidos contêineres responsáveis por encapsular o comportamento dos medidores. Foram implementados quatro contêineres distintos, correspondentes às combinações entre tipo de medidor e protocolo de comunicação: medidor de água via HTTP, medidor de água via CoAP, medidor de energia via HTTP e medidor de energia via CoAP.

Os medidores baseados em HTTP foram desenvolvidos em *Python*, enquanto os medidores que utilizam o protocolo CoAP foram implementados em *Node.js*, devido ao melhor suporte e disponibilidade de bibliotecas específicas para esse protocolo no momento do desenvolvimento.

No ambiente de execução, o nó primário é responsável por receber dados utilizando os protocolos CoAP e HTTP. Independentemente do protocolo de origem, as mensagens são convertidas para HTTP antes de serem encaminhadas aos nós de névoa, padronizando o tráfego interno do sistema.

Dentro de um nó de névoa, a comunicação entre a subcamada de protocolos e a subcamada de processamento ocorre por meio de *WebSocket*, mantendo a conexão aberta e reduzindo o custo de abertura e fechamento contínuo de conexões. De acordo com Murley et al. (Murley et al., 2021), conexões WebSocket permitem comunicação bidirecional persistente com menor overhead em comparação a abordagens baseadas em requisições HTTP repetidas, o que contribui para reduzir a sobrecarga e melhorar o tempo de resposta.

[1] Disponível em: `https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption`.

[2] Disponível em: `https://data.hellenicdataservice.gr/dataset/78776f38-a58b-4a2a-a8f9-85b964fe5c95`.

Após o processamento, os dados são estruturados em formato Comma-Separated Values (CSV), a partir das mensagens recebidas em JavaScript Object Notation (JSON). A transmissão dos arquivos do nó de névoa para o nó agregador é realizada via SFTP. O nó agregador consolida os arquivos provenientes de diferentes nós e os encaminha, também por SFTP, à camada de nuvem para armazenamento na zona de entrada (*landing zone*) do ambiente HPCC *Systems*.

5.4 DISTRIBUIÇÃO ENTRE NÉVOAS E BALANCEAMENTO DE CARGA

Nos experimentos, os 70 dispositivos foram distribuídos entre dois domínios de névoa distintos, representando diferentes regiões no cenário de cidade inteligente. Cada tipo de dispositivo foi instanciado diversas vezes, totalizando 21 medidores de energia via *HTTP*, 17 medidores de energia via *CoAP*, 15 medidores de água via *HTTP* e 17 medidores de água via *CoAP*. Esses dispositivos transmitiram leituras reais de consumo em formato JSON, com frequência horária, diretamente para a camada de névoa.

A distribuição inicial entre as névoas foi propositalmente desigual, a fim de avaliar o comportamento do sistema sob diferentes condições de carga:

- **Névoa A**: 40 dispositivos (12 medidores de energia via *HTTP*, 10 via *CoAP*, 9 medidores de água via *HTTP* e 9 via *CoAP*).
- **Névoa B**: 30 dispositivos (9 medidores de energia via *HTTP*, 7 via *CoAP*, 6 medidores de água via *HTTP* e 8 via *CoAP*).

Quando uma das névoas atingia maior taxa de utilização ou apresentava desequilíbrio na distribuição de especializações entre os nós disponíveis, o mecanismo de balanceamento era acionado. Nesse caso, o nó primário da névoa mais sobrecarregada redirecionava parte das requisições para a outra névoa, realizando a conversão para *HTTP* antes do envio, independentemente do protocolo de origem (*HTTP* ou *CoAP*). Além do balanceamento entre diferentes domínios de névoa, o nó primário também é responsável por gerenciar a distribuição dos dispositivos entre os nós de névoa disponíveis dentro do mesmo domínio. Caso um nó apresente maior quantidade de dispositivos associados ou maior carga de processamento, o nó primário pode redistribuir parte desses dispositivos para outros nós da mesma névoa, buscando manter uma distribuição mais equilibrada dos recursos computacionais.

O sistema também monitora continuamente a disponibilidade dos nós de névoa. Caso um nó deixe de responder às verificações de conectividade, ele é automaticamente considerado indisponível e removido da lista de nós ativos. Nos experimentos realizados, o tempo máximo configurado para detecção de falha foi de 30 segundos. Após esse período sem resposta, o nó é removido do conjunto de nós disponíveis, evitando que novas requisições sejam direcionadas a ele.

Nos experimentos também foi considerado o cenário de falha de um nó primário. Os nós primários realizam verificações periódicas de disponibilidade a cada 30 segundos para confirmar que os demais domínios permanecem ativos. Caso um nó primário deixe de responder durante esse intervalo, ele é considerado indisponível, permitindo que o nó primário de outro domínio assuma temporariamente a coordenação. Nesse caso, são enviadas mensagens de controle aos dispositivos conectados, instruindo-os a redirecionar suas transmissões para o novo nó primário disponível. Esse mecanismo permite manter o fluxo de dados mesmo na ausência do nó primário originalmente responsável pelo domínio.

Durante os testes, foram simuladas 48 horas de operação, resultando em um total de 3.360 leituras (70 dispositivos $\times$ 48 envios cada), com medições horárias agregadas em janelas de consumo. No nó agregador, as medições de cada domínio de névoa foram consolidadas em um único arquivo por dia e transmitidas para o ambiente HPCC *Systems*, sem ocorrência de falhas durante a execução dos contêineres.

5.5 BANCO DE DADOS E LINGUAGEM DE CONSULTA DE DADOS

Nos experimentos realizados, foi utilizado um banco de dados local nos nós de névoa para armazenamento temporário das informações recebidas dos dispositivos. Esse armazenamento é utilizado durante o processamento local dos dados antes de sua consolidação e encaminhamento para as camadas superiores da arquitetura.

Para essa finalidade, foi adotado o banco de dados *MongoDB*. A escolha desse banco se deu principalmente pela sua flexibilidade no armazenamento de dados em formato JSON, característica que facilita o tratamento das mensagens provenientes dos dispositivos. Além disso, o *MongoDB* permite a inserção de registros heterogêneos sem a necessidade de um esquema rígido, o que se mostrou adequado para lidar com diferentes tipos de medições e estruturas de dados utilizadas nos testes.

A comunicação entre a subcamada de serviços e a subcamada de processamento foi implementada utilizando a linguagem de consulta *GraphQL*. Essa tecnologia permite que as aplicações especifiquem exatamente quais campos e registros devem ser retornados, reduzindo a transferência de dados desnecessários e diminuindo a sobrecarga de processamento.

Outra característica relevante do *GraphQL* é a possibilidade de realizar consultas a múltiplas fontes de dados de forma unificada, agregando resultados distintos em uma única resposta. Dessa forma, a subcamada de serviços consegue solicitar apenas as informações necessárias ao processamento, recebendo uma resposta já filtrada e adequada ao contexto da aplicação.

Cabe destacar que tanto o *MongoDB* quanto o *GraphQL* foram utilizados como tecnologias de apoio na implementação dos testes. A arquitetura proposta não depende especificamente dessas ferramentas, podendo utilizar outras tecnologias equivalentes

que atendam aos requisitos funcionais definidos para o armazenamento e consulta de dados.

5.6 HPCC SYSTEMS NA CAMADA DE NUVEM

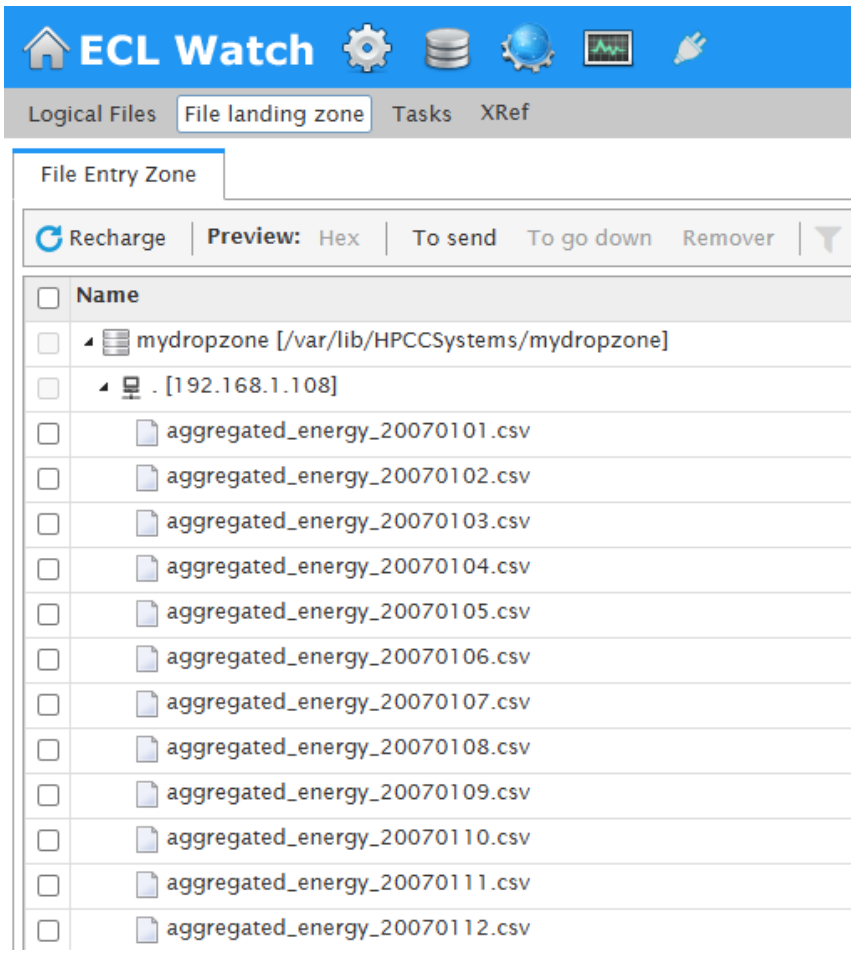
Na fase de testes, os dados provenientes dos nós de névoa de cada domínio foram enviados para seus respectivos nós agregadores, cada um responsável de forma exclusiva pela consolidação das informações dentro do domínio ao qual pertence. Nesse processo, os arquivos no formato `.csv` gerados pelos nós de névoa foram unificados em um único arquivo consolidado. Quando necessário, o nó agregador realizou operações adicionais, como a soma de valores correspondentes.

Após a consolidação, cada nó agregador transmitiu o arquivo final para a camada de nuvem, implementada com o HPCC *Systems*. Os arquivos recebidos no formato `.csv`, que é um dos formatos suportados de forma nativa pela plataforma, foram armazenados e processados para análise posterior, representando o resultado da integração e do pré-processamento realizados nas camadas inferiores da arquitetura.

Esses arquivos foram disponibilizados na zona de entrada da plataforma (*landing zone*), permanecendo prontos para processamento e consultas conforme as necessidades da aplicação.

A Figura 5 apresenta o ambiente ECL *Watch*, exibindo os arquivos agregados de consumo de energia já no formato consolidado por data, prontos para tratamento e análise no ambiente de nuvem.

Figura 5 – Arquivos consolidados na *landing zone* do HPCC *Systems*.



Fonte: Do autor.

6 RESULTADOS

Esta seção apresenta os valores medidos a partir dos testes realizados na arquitetura proposta. Para cada conjunto de valores coletados, são descritos o método de medição e uma análise interpretativa dos resultados, destacando seu significado e discutindo seu impacto no desempenho do sistema. Além disso, sempre que aplicável, os resultados são comparados com aqueles obtidos em outras arquiteturas ou trabalhos relacionados, a fim de contextualizar o desempenho e evidenciar os avanços proporcionados pela solução proposta.

6.1 VALORES MEDIDOS

Os valores apresentados nas Tabelas 4 e 5 foram obtidos a partir de *timestamps* registrados nos pontos de entrada e saída de cada componente, bem como de medições de consumo de memória realizadas durante o envio das mensagens.

Inicialmente são apresentadas as medições de latência entre os principais componentes da arquitetura. Essas medições permitem observar o tempo necessário para que uma mensagem percorra o fluxo completo do sistema, desde o envio pelo dispositivo até o armazenamento final na nuvem.

A Tabela 4 apresenta as latências mínimas, máximas e médias registradas em cada segmento da comunicação. Os valores incluem o envio de dados do medidor para o nó primário, o encaminhamento do nó primário para os nós de névoa, a comunicação entre diferentes domínios de névoa e o envio final dos dados consolidados para a plataforma HPCC *Systems*.

Quando ocorre indisponibilidade de um nó de névoa ou desequilíbrio de carga, o nó primário pode redirecionar requisições para outro domínio de névoa. Nesse processo, o nó primário inicialmente estabelece comunicação com o nó primário do domínio remoto para verificar a disponibilidade de recursos. Após a confirmação, as requisições passam a ser encaminhadas para os nós de névoa do outro domínio. Em situações de falha do nó primário, o nó primário de outro domínio pode assumir temporariamente a coordenação e enviar mensagens de controle aos dispositivos, instruindo-os a redirecionar suas transmissões para o novo nó primário disponível. Os tempos de comunicação entre nós primários e o envio dessas mensagens de redirecionamento também foram considerados nas medições apresentadas na Tabela 4.

A análise das latências evidencia que o maior tempo de transmissão ocorre entre o Agregador e o HPCC *Systems*, etapa que concentra a maior parte da latência total.

Além das medições de latência, também foi avaliado o consumo de memória dos principais componentes da arquitetura durante a execução dos experimentos. Essas medições permitem observar o impacto do processamento distribuído nos recursos

Tabela 4 – Latências mínimas, máximas e médias medidas

Caminho	**Mínimo (ms)**	**Máximo (ms)**	**Média (ms)**
Comunicação dentro do domínio de névoa			
Medidor → Nó Primário	456	2455	1455
Nó Primário → Nó de Névoa	1	3	2
Nó de Névoa → Agregador	2	3	2,5
Agregador → Nuvem (HPCC *Systems*)	600	1600	1100
Comunicação entre domínios de névoa			
Nó Primário → Nó Primário (outra névoa)	3	9	6
Nó Primário → Medidor (outra névoa)	25	85	48

Fonte: Do autor.

computacionais utilizados pelos diferentes elementos do sistema.

A Tabela 5 apresenta o consumo médio de memória por componente do sistema, obtido durante a execução contínua de 680.000 requisições, o que garante consistência estatística para os valores apresentados.

Tabela 5 – Consumo médio de memória por componente

Componente	**Memória (MiB)**
Nó Primário	37,37
Agregador	40,14
Nó de Névoa (Energia)	276,60
Nó de Névoa (Água)	273,42
Simulador de Medidor HTTP	19,97
Simulador de Medidor CoAP	13,58

Fonte: Do autor.

Em relação ao consumo de memória, observa-se que os nós de névoa apresentam maior demanda devido às operações de pré-processamento e armazenamento temporário, enquanto os componentes de controle, como o nó primário de névoa e o nó agregador, consomem menos recursos.

6.2 DISCUSSÃO DOS RESULTADOS

Os resultados obtidos demonstram que a arquitetura proposta apresenta um baixo consumo de recursos quando comparada com o estudo de D'Agostino *et al.*, no estudo relata-se que a execução de um ambiente de névoa implementado em Docker Compose consumia aproximadamente 1,2 GB de memória com apenas um contêiner ativo (D'Agostino; Longo; Palmieri, 2025). Mesmo com todos os componentes em funcionamento, o sistema proposto manteve o consumo total em cerca de 374 MiB, o que corresponde a apenas 31% do valor relatado naquele estudo, representando uma redução aproximada de 69% no uso de memória.

Quando comparado a trabalhos recentes, o sistema proposto complementa e estende abordagens existentes. Rahman e Hussain (Rahman; Hussain, 2024) exploraram a interoperabilidade semântica em um *testbed* com múltiplos domínios de névoa, enquanto a arquitetura aqui apresentada enfatiza a interoperabilidade de protocolos e a composição modular. Oliveira *et al.* (Oliveira; Santos; Mendes, 2024) aprimoraram o posicionamento modular de aplicações, mas sem coordenação entre diferentes névoas; nossa solução amplia essa capacidade por meio de redirecionamento adaptativo de requisições entre nós primários. Já Boudieb *et al.* (Boudieb; Ayed; Harous, 2024) abordaram o balanceamento dinâmico de carga com *deep reinforcement learning*, ao passo que a presente proposta alcança uma distribuição adaptativa mais simples, baseada em coordenação autônoma e que consome menos recursos.

7 CONCLUSÃO

Este trabalho apresenta o desenvolvimento e a avaliação de uma arquitetura modular voltada a ambientes de computação em névoa, projetada para suportar serviços configuráveis e possibilitar a comunicação entre diferentes domínios de névoa. A arquitetura proposta foi estruturada em três camadas principais: dispositivos, névoa e nuvem. A camada de dispositivos é responsável pela geração dos dados, enquanto a camada de névoa realiza o processamento intermediário e a coordenação das requisições antes do envio das informações para a camada de nuvem. Cada domínio corresponde a uma instância de névoa, podendo existir múltiplos domínios operando de forma independente. Dentro de cada domínio, múltiplos nós de névoa podem ser implantados com diferentes especializações de serviço, como monitoramento de água ou de energia. Internamente, cada nó de névoa é organizado em três subcamadas funcionais, protocolos, processamento e serviços, responsáveis pela comunicação, pelo gerenciamento interno dos dados e pela execução das aplicações.

Os experimentos realizados com medidores de energia e água demonstraram que a arquitetura mantém um fluxo de dados consistente desde a borda até a nuvem. A abstração de protocolos, aplicada entre HTTP e CoAP, simplificou a interoperabilidade entre dispositivos distintos, enquanto o uso da plataforma HPCC *Systems* na camada de nuvem possibilitou a agregação e a análise dos dados em larga escala. Além disso, duas instâncias de névoa puderam operar de forma coordenada e autônoma, realizando balanceamento dinâmico de carga entre os nós responsáveis pelos serviços de água e de energia. Esse balanceamento ocorreu tanto pela redistribuição de requisições entre nós de uma mesma névoa quanto pelo redirecionamento de requisições entre diferentes domínios de névoa quando identificado desequilíbrio de carga ou limitação de recursos. Em cenários de falha de um nó primário, outro nó primário disponível pôde assumir temporariamente a coordenação, enviando mensagens de controle aos dispositivos para redirecionar suas transmissões, garantindo a continuidade do fluxo de dados. A coordenação entre nós primários implementa uma forma simplificada de tomada de decisão local, inspirada nos conceitos do OpenFog, contribuindo para o processamento distribuído e para a expansão flexível da infraestrutura (IEEE. . . , 2018). A arquitetura também apresentou baixo consumo de recursos, com aproximadamente 374 MiB no total, o que representa uma redução de cerca de 69% em relação ao consumo relatado em implementações baseadas em Docker Compose, reforçando o caráter leve e eficiente da solução.

Durante o desenvolvimento e os experimentos apresentados neste trabalho, diferentes nós de névoa foram configurados com serviços específicos para consulta e processamento de dados, como serviços voltados ao monitoramento de água e outros dedicados ao monitoramento de energia, evidenciando a capacidade da arquitetura

de suportar múltiplos serviços distribuídos entre os nós de névoa. Como perspectivas futuras, recomenda-se a integração de novos protocolos de comunicação, como Message Queuing Telemetry Transport (MQTT), bem como a adoção de mecanismos de autenticação e criptografia ponta a ponta.

REFERÊNCIAS

BARRETT, Daniel J.; SILVERMAN, Richard E.; BYRNES, Robert G. **SSH, The Secure Shell: The Definitive Guide**. 2. ed. Sebastopol, CA: O'Reilly Media, 2005. ISBN 978-0596008956.

BORMANN, Carsten; CASTELLANI, Angelo P.; SHELBY, Zach. CoAP: An Application Protocol for Billions of Tiny Internet Nodes. **IEEE Internet Computing**, v. 16, n. 2, p. 62–67, mar. 2012.

BOUDIEB, A.; AYED, M.; HAROUS, S. Microservice Instances Selection and Load Balancing in Fog Computing. **Future Generation Computer Systems**, v. 154, p. 510–523, 2024.

D'AGOSTINO, R.; LONGO, M.; PALMIERI, G. A Scalable Fog Computing Solution for Industrial Predictive Maintenance. **Electronics**, v. 14, n. 2, p. 1–18, 2025.

GOURLEY, David; TOTTY, Brian; SAYER, Marjorie; REDDY, Sailu; AGGARWAL, Anshu. **HTTP: The Definitive Guide**. Sebastopol, CA: O'Reilly Media, 2002. ISBN 978-1565925090.

HELLENIC DATA SERVICE. **Water Consumption Dataset**. [S.l.*:* s.n.]. `https://data.hellenicdataservice.gr/dataset/78776f38-a58b-4a2a-a8f9-85b964fe5c95`. Acesso em: 11 ago. 2025.

HERNÁNDEZ, Héctor et al. **Individual household electric power consumption Data Set**. [S.l.*:* s.n.]. `https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption`. Acesso em: 11 ago. 2025.

IEEE Standard for Adoption of OpenFog Reference Architecture for Fog Computing. [S.l.*:* s.n.], 2018. p. 1–176.

KHAN, H. M.; KHAN, A.; JABEEN, F.; RAHMAN, A. U. Privacy preserving data aggregation with fault tolerance in fog-enabled smart grids. **Sustainable Cities and Society**, v. 64, p. 102522, 2021.

MARKAKIS, E. K.; SIDERIS, A.; MASTORAKIS, G.; MAVROMOUSTAKIS, C. X.; PALLIS, E.; CHARALABIDIS, Y. EXEGESIS: Extreme Edge Resource Harvesting for a Virtualized Fog Environment. **IEEE Communications Magazine**, v. 55, n. 7, p. 173–179, jul. 2017.

MELL, P.; GRANCE, T. The NIST Definition of Cloud Computing. **NIST Special Publication 800-145**, v. 800, n. 145, p. 1–7, 2011.

MURLEY, P.; MA, Z.; MASON, J.; BAILEY, M.; KHARRAZ, A. WebSocket Adoption and the Landscape of the Real-Time Web. **Proceedings of the Web Conference (WWW)**, v. 2021, n. 1, p. 1192–1203, 2021.

OLIVEIRA, J.; SANTOS, A.; MENDES, L. Enhancing Modular Application Placement in Hierarchical Fog Computing. **Computer Communications**, v. 222, p. 180–194, 2024.

RAHMAN, M.; HUSSAIN, M. Testbed Analysis of Multi-Fog Architecture for Interoperable IoT. **Wireless Networks**, v. 30, p. 451–463, 2024.

RAHMANI, Amir M.; GIA, Tuan Nguyen; NEGASH, Behailu; ANZANPOUR, Ali; AZIMI, Iman; LAGERSPETZ, Mikael; LILJEBERG, Pasi. Exploiting smart e-Health gateways at the edge of healthcare Internet-of-Things: A fog computing approach. **Future Generation Computer Systems**, v. 78, p. 641–658, 2018.

SILVEIRA, Rubens Prates da. **GraphQL**: A nova alternativa para criação de APIs. São Paulo: Casa do Código, 2019. ISBN 978-6586057956.

TANENBAUM, Andrew S.; BOS, Herbert. **Modern Operating Systems**. 4. ed. Boston: Pearson, 2015. ISBN 978-0133591620.

TANENBAUM, Andrew S.; WETHERALL, David J. **Computer Networks**. 5. ed. Boston: Pearson, 2011. ISBN 978-0132126953.

TAYLOR, Richard. **Mastering HPCC Systems: Platform Overview and History**. [S.l.]: Function, 2022. Kindle Edition.

TSENG, Chih-Lin; LIN, Fu-Jen. Extending Scalability of IoT/M2M Platforms with Fog Computing. In*:* 2018 IEEE 4th World Forum on Internet of Things (WF-IoT). [S.l.*:* s.n.], 2018. p. 825–830.

WANG, Quan; DING, Weiping; XU, Xiaolong; WU, Fei; JIA, Weijia. WebSocket-based Real-time Communication for Industrial Automation. In*:* PROCEEDINGS of the 2013 IEEE International Conference on Industrial Technology (ICIT). [S.l.*:* s.n.], 2013. p. 1080–1085.